

To use the Authentication with the Authorization builder:

```
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
}).AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = builder.Configuration["Jwt:Issuer"],
        ValidAudience = builder.Configuration["Jwt:Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))
    }
});

builder.Services.AddAuthentication();

builder.Services.AddAuthorizationBuilder().AddPolicy("CanManageUsers", policy =>
    policy.RequireClaim("Permission", "ManageUsers")
).AddPolicy("CanViewReports", policy =>
    policy.RequireClaim("Permission", "ViewReports").RequireRole("Admin", "Manager")
);
```

=====

To seed the data to the DB:

```
static async Task SeedDataAsync(IServiceProvider services)
{
    var roleManager = services.GetRequiredService<RoleManager<IdentityRole>>();
    var userManager = services.GetRequiredService<UserManager<ApplicationUser>>();

    // Seed roles
    string[] roleNames = { "Admin", "Manager", "User" };
    foreach (var roleName in roleNames)
    {
        if (!await roleManager.RoleExistsAsync(roleName))
            await roleManager.CreateAsync(new IdentityRole(roleName));
    }

    // Seed admin user
    var adminEmail = "admin@yourdomain.com";
    var adminUser = await userManager.FindByEmailAsync(adminEmail);

    if (adminUser == null)
    {
        adminUser = new ApplicationUser
        {
            UserName = adminEmail,
            Email = adminEmail,
            FirstName = "System",
            LastName = "Admin",
            EmailConfirmed = true,
        }
    }
}
```

```

        IsActive = true
    };

    var result = await userManager.CreateAsync(adminUser, "Admin@123!");
    if (result.Succeeded)
    {
        await userManager.AddToRoleAsync(adminUser, "Admin");
        // Add admin claims
        await userManager.AddClaimAsync(adminUser, new Claim("Permission",
"ManageUsers"));
        await userManager.AddClaimAsync(adminUser, new Claim("Permission",
"ViewReports"));
    }
}
}

```

=====

To use the service with the seed data method:

```

using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    await SeedDataAsync(services);
}

```

=====

To define the custom requirement handler:

```
namespace ASP.NET_Core_Authentication___Authorization.Handlers
{
    public class ClaimRequirement : IAuthorizationRequirement
    {
        public string ClaimType { get; }
        public string ClaimValue { get; }

        public ClaimRequirement(string type, string value)
        {
            ClaimType = type;
            ClaimValue = value;
        }
    }

    public class ClaimRequirementHandler : AuthorizationHandler<ClaimRequirement>
    {
        protected override Task HandleRequirementAsync(
            AuthorizationHandlerContext context,
            ClaimRequirement requirement)
        {
            var claim = context.User.Claims.FirstOrDefault(c =>
                c.Type == requirement.ClaimType && c.Value == requirement.ClaimValue);

            if (claim != null)
                context.Succeed(requirement);

            return Task.CompletedTask;
        }
    }
}
```

=====

To register with the Program:

```
// Register in Program.cs:
builder.Services.AddSingleton<IAuthorizationHandler, ClaimRequirementHandler>();
=====
```

The Auth Controller for Log in, Register, and Generate the token:

```
namespace ASP.NET_Core_Authentication___Authorization.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AuthController : ControllerBase
    {
        private readonly UserManager<ApplicationUser> _userManager;
        private readonly SignInManager<ApplicationUser> _signInManager;
        private readonly IConfiguration _configuration;
        private readonly ILogger<AuthController> _logger;

        public AuthController(
            UserManager<ApplicationUser> userManager,
            SignInManager<ApplicationUser> signInManager,
            IConfiguration configuration,
            ILogger<AuthController> logger)
        {
            _userManager = userManager;
            _signInManager = signInManager;
            _configuration = configuration;
            _logger = logger;
        }
    }
}
```

```

        ILogger<AuthController> logger)
    {
        _userManager = userManager;
        _signInManager = signInManager;
        _configuration = configuration;
        _logger = logger;
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginRequest model)
    {
        var user = await _userManager.FindByEmailAsync(model.Email);

        if (user == null || !await _userManager.CheckPasswordAsync(user,
model.Password))
        {
            _logger.LogWarning("Failed login attempt for email: {Email}",
model.Email);
            return Unauthorized(new { message = "Invalid credentials" });
        }

        if (!user.IsActive)
        {
            return BadRequest(new { message = "Account is disabled" });
        }

        // Generate JWT with claims
        var token = await GenerateJwtTokenAsync(user);

        // Update last login
        user.LastLogin = DateTime.UtcNow;
        await _userManager.UpdateAsync(user);

        _logger.LogInformation("User {UserId} logged in successfully", user.Id);

        return Ok(new
        {
            token = new JwtSecurityTokenHandler().WriteToken(token),
            expiration = token.ValidTo,
            user = new { user.Id, user.Email, user.UserName }
        });
    }

    [HttpPost("register")]
    public async Task<IActionResult> Register([FromBody] RegisterRequest model)
    {
        var user = new ApplicationUser
        {
            UserName = model.Email,
            Email = model.Email,
            FirstName = model.FirstName,
            LastName = model.LastName,
            EmailConfirmed = true
        };

        var result = await _userManager.CreateAsync(user, model.Password);

        if (!result.Succeeded)
        {
            return BadRequest(result.Errors);
        }
    }

```

```

        // Assign default role
        await _userManager.AddToRoleAsync(user, "User");

        _logger.LogInformation("New user registered: {UserId}", user.Id);

        return StatusCode(201, new { message = "User created successfully" });
    }

    private async Task<JwtSecurityToken> GenerateJwtTokenAsync(ApplicationUser user)
    {
        var claims = new List<Claim>
        {
            new Claim(JwtRegisteredClaimNames.Sub, user.Id),
            new Claim(JwtRegisteredClaimNames.Email, user.Email ?? ""),
            new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
            new Claim("name", $"{user.FirstName} {user.LastName}")
        };

        // Add user roles as claims
        var roles = await _userManager.GetRolesAsync(user);
        foreach (var role in roles)
        {
            claims.Add(new Claim(ClaimTypes.Role, role));
        }

        // Add custom user claims from database
        var userClaims = await _userManager.GetClaimsAsync(user);
        claims.AddRange(userClaims);

        var key = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]!));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
        var expires = DateTime.UtcNow.AddMinutes(
            Convert.ToDouble(_configuration["Jwt:ExpireMinutes"]))
        );

        return new JwtSecurityToken(
            issuer: _configuration["Jwt:Issuer"],
            audience: _configuration["Jwt:Audience"],
            claims: claims,
            expires: expires,
            signingCredentials: creds
        );
    }
}

```

=====

The user management controller:

```
namespace ASP.NET_Core_Authentication___Authorization.Controllers
{
    [Authorize(Roles = "Admin")]
    [Route("api/admin/[controller]")]
    [ApiController]
    public class UserManagementController : ControllerBase
    {
        private readonly UserManager<ApplicationUser> _userManager;
        private readonly RoleManager<IdentityRole> _roleManager;
        private readonly ILogger<UserManagementController> _logger;

        public UserManagementController(
            UserManager<ApplicationUser> userManager,
            RoleManager<IdentityRole> roleManager,
            ILogger<UserManagementController> logger)
        {
            _userManager = userManager;
            _roleManager = roleManager;
            _logger = logger;
        }

        // === ROLE MANAGEMENT ===
        [HttpPost("roles")]
        public async Task<IActionResult> CreateRole([FromBody] string roleName)
        {
            if (await _roleManager.RoleExistsAsync(roleName))
                return BadRequest(new { message = "Role already exists" });

            var result = await _roleManager.CreateAsync(new IdentityRole(roleName));

            if (!result.Succeeded)
                return BadRequest(result.Errors);

            _logger.LogInformation("Created role: {RoleName}", roleName);
            return Ok(new { message = "Role created" });
        }

        [HttpDelete("roles/{roleName}")]
        public async Task<IActionResult> DeleteRole(string roleName)
        {
            var role = await _roleManager.FindByNameAsync(roleName);
            if (role == null) return NotFound();

            var result = await _roleManager.DeleteAsync(role);
            if (!result.Succeeded) return BadRequest(result.Errors);

            _logger.LogInformation("Deleted role: {RoleName}", roleName);
            return Ok(new { message = "Role deleted" });
        }

        // === USER ROLE ASSIGNMENT ===
        [HttpPost("users/{userId}/roles")]
        public async Task<IActionResult> AssignRoleToUser(
            string userId,
            [FromBody] string roleName)
        {
            var user = await _userManager.FindByIdAsync(userId);
            if (user == null) return NotFound();
        }
    }
}
```

```

        if (!await _roleManager.RoleExistsAsync(roleName))
            return BadRequest(new { message = "Role does not exist" });

        var result = await _userManager.AddToRoleAsync(user, roleName);
        if (!result.Succeeded) return BadRequest(result.Errors);

        _logger.LogInformation("Assigned role {RoleName} to user {UserId}",
roleName, userId);
        return Ok(new { message = "Role assigned" });
    }

    [HttpDelete("users/{userId}/roles/{roleName}")]
    public async Task<IActionResult> RemoveRoleFromUser(
        string userId,
        string roleName)
    {
        var user = await _userManager.FindByIdAsync(userId);
        if (user == null) return NotFound();

        var result = await _userManager.RemoveFromRoleAsync(user, roleName);
        if (!result.Succeeded) return BadRequest(result.Errors);

        _logger.LogInformation("Removed role {RoleName} from user {UserId}",
roleName, userId);
        return Ok(new { message = "Role removed" });
    }

    // === CLAIMS MANAGEMENT ===

    [HttpPost("users/{userId}/claims")]
    public async Task<IActionResult> AddClaimToUser(
        string userId,
        [FromBody] ClaimRequest claim)
    {
        var user = await _userManager.FindByIdAsync(userId);
        if (user == null) return NotFound();

        var claims = await _userManager.GetClaimsAsync(user);
        var newClaim = new Claim(claim.Type, claim.Value);

        if (claims.Any(c => c.Type == newClaim.Type && c.Value == newClaim.Value))
        {
            return Conflict();
        }

        var result = await _userManager.AddClaimAsync(user, newClaim);

        if (!result.Succeeded) return BadRequest(result.Errors);

        _logger.LogInformation("Added claim {ClaimType}={ClaimValue} to user
{UserId}",
            claim.Type, claim.Value, userId);
        return Ok(new { message = "Claim added" });
    }

    [HttpDelete("users/{userId}/claims")]
    public async Task<IActionResult> RemoveClaimFromUser(
        string userId,
        [FromBody] ClaimRequest claim)

```

```

    {
        var user = await _userManager.FindByIdAsync(userId);
        if (user == null) return NotFound();

        var existingClaims = await _userManager.GetClaimsAsync(user);
        var claimToRemove = existingClaims.FirstOrDefault(c =>
            c.Type == claim.Type && c.Value == claim.Value);

        if (claimToRemove == null)
            return NotFound(new { message = "Claim not found on user" });

        var result = await _userManager.RemoveClaimAsync(user, claimToRemove);
        if (!result.Succeeded) return BadRequest(result.Errors);

        _logger.LogInformation("Removed claim {ClaimType}={ClaimValue} from user
{UserId}",
            claim.Type, claim.Value, userId);
        return Ok(new { message = "Claim removed" });
    }

    // === GET USER DETAILS WITH ROLES & CLAIMS ===

    [HttpGet("users/{userId}")]
    public async Task<IActionResult> GetUserDetails(string userId)
    {
        var user = await _userManager.FindByIdAsync(userId);
        if (user == null) return NotFound();

        var roles = await _userManager.GetRolesAsync(user);
        var claims = await _userManager.GetClaimsAsync(user);

        return Ok(new
        {
            user.Id,
            user.Email,
            user.UserName,
            user.FirstName,
            user.LastName,
            user.IsActive,
            Roles = roles,
            Claims = claims.Select(c => new { c.Type, c.Value })
        });
    }
}

```

=====

To use the authentication and authorization with other controllers:

```

namespace ASP.NET_Core_Authentication__Authorization.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductController : ControllerBase
    {
        // Public endpoint
        [HttpGet]
        public async Task<IActionResult> GetAll()
        {
            return Ok();
        }
    }
}

```

```

// Authenticated users only
[Authorize]
[HttpGet("{id}")]
public async Task<IActionResult> GetById(string id)
{
    return Ok();
}

// Admins only
[Authorize(Roles = "Admin")]
[HttpPost]
public async Task<IActionResult> Create()
{
    return Ok();
}

// Policy-based authorization
[Authorize(Policy = "CanManageUsers")]
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(string id)
{
    return Ok();
}
}
}

=====

```